

Architecture & Trade-offs

In-memory account ledger | Implementation reviewed: e587cfc | 7 October 2026

This document describes the implemented core and labels production proposals explicitly. It does not claim that the exercise is a deployable banking ledger. Source: [repository at the reviewed revision](#).

1. Append-only at scale

Current architecture. `Ledger.ingest()` validates commands, appends frozen postings and decisions, and derives balances and holds by scanning history. Daily snapshots preserve the view known at close; restated views use later information. `finish()` computes interest and closes the batch. Runtime immutability provides understandable replay, not durability or tamper-proof storage.

What breaks first at 100x? For fixed accounts and window length, repeated scans are the first expected latency bottleneck. Duplicate detection scans decisions; every authorization query rebuilds the full hold projection. After every accepted event, `#assessThrough()` scans all closed days and accounts, and each balance query scans all postings. With N events, P postings, A accounts and D days, that catch-up work is $O(D \times A \times (P + A))$ per event; with fixed A/D and P proportional to N , replay can grow quadratically. Increasing accounts as well makes the global scans worse.

An actual local probe of 10 versus 1,000 unique credits completed at both sizes: median 0.192 ms versus 46.710 ms. This is a 100x input increase, not evidence of a failure threshold. It used two accounts, a fixed six-day horizon, Day 6 credits, Node 25.3.0 on macOS arm64, one warm-up per size and seven measured runs; constructor, ingestion and finalization were timed. Event generation, explicit GC and reporting were excluded. Sub-millisecond baseline noise and the absence of holds or backdated debits limit extrapolation. [Reproducible evidence](#).

Where state grows. `#entries` and `#decisions` retain every posting and attempt, including rejected retries, without an event-count bound. There are at most six snapshots in this batch, but each can copy every known authorization, including terminal states. Interest rows are bounded by accounts times days. Getter copies, `createReport()` and text rendering additionally materialize the full history. Continuing across periods would multiply retained snapshots; deleting them alone would not bound the source journal.

Cheapest structural change. Keep the journal, but maintain rebuildable indexes: event-ID set, authorization map plus active-hold total per account, assessed-day set, and per-account daily balance buckets. Recalculate only the affected account from its earliest changed value date. Preserve reception order and compare projected results against replay. This trades extra derived state and consistency checks for fewer global scans; it does not solve unbounded history.

To bound resident history, the next step is a durable segmented journal with sequence-numbered checkpoints and archived segments. Evict only after durable acknowledgement and proven reconstruction. A checkpoint is an acceleration structure, not permission to destroy records subject to retention or legal holds. Sharding is premature before removing the global scans.

2. Value-dated entries in production

Operational surface. The core separates supplied booking day, observed recording day and economic value day. Production needs real receipt timestamps, a controlled banking calendar and close watermarks. A late entry can change historical balances, fees, accrued income, statements, customer redress and general-ledger reconciliation. Previously issued statements and regulatory returns need a controlled correction process, not silent replacement. Historical authorization decisions must remain explainable using information available when they were made.

The existing fee policy treats the affected historical day as the assessment/value day and retains assessed fees when principal is reversed. Those are declared exercise choices, not legal conclusions. The current model lacks reversal reason codes, so it cannot distinguish a valid commercial reversal from correction of the bank's own error.

UAE regulatory surface. For a UAE-licensed bank serving consumers, CBUAE Consumer Protection Standards 5.1.1.37-40 require handling institutional errors, correcting affected consumers and refunding deductions or direct costs caused by such errors. A blanket policy retaining consequential fees would therefore be unsuitable for that case. The remedy can remain append-only through linked refund/adjustment entries. [1]

CBUAE Internal Control Standards, Article 2(2), cover account reconciliation, segregation of duties and dual controls. The architecture must connect customer-ledger corrections to finance review and reconciliation evidence. [2] Cabinet Resolution 134/2025, Article 25, requires reconstructable transaction records, prompt availability to authorities and at least five years of retention, with record-specific start triggers; covered due-diligence/account records use the latest applicable trigger. An arbitrary age-based journal purge is not adequate. [3]

One control before go-live: a controlled backdate approval gate. Any posting crossing a closed-day watermark enters a pending queue. Require a reason, source evidence, original reference, receipt timestamp and policy version. Generate a preview of affected balances, fees, interest and issued outputs; a separate authorized reviewer approves or rejects it. The approved request and linked adjustments commit atomically with an idempotency key. Finance then reconciles the change and records whether customer redress or report correction is needed. Preserve the request, approval and both before/after views. Product and Compliance own permitted backdating windows; no unexplained fixed number of days is invented here.

This gate is a proposed implementation control grounded in those obligations; the sources do not prescribe this exact workflow. It cannot replace durable accounting, consumer communications, AML monitoring or product approval. The exercise's pricing and interest model are not evidence of approved UAE product terms; an Islamic product would additionally need applicable Sharia governance review.

Official references

[1] CBUAE Consumer Protection Standards, Article 5, clauses 5.1.1.37-40.

[2] CBUAE Internal Controls, Compliance and Internal Audit Standards, Article 2.

[3] Cabinet Resolution 134/2025, Article 25: record keeping. Sources checked 7 October 2026. Applicability and retention schedules require the bank's Compliance review.

3. Authorization lifecycle

What the model actually supports. The states are ACTIVE, SETTLED and REJECTED. An insufficient-funds request is rejected at inception; it never creates an active hold. Once ACTIVE, the only implemented terminal transition is a matching final settlement. There is no cancellation, expiry, administrative release or replacement event. `finish()` leaves active holds active. A principal reversal does not cancel a hold. Process termination loses memory; it is not a valid lifecycle transition.

Consequently, the current model has **no non-settlement termination for an active authorization**. Describing any of the paths below as implemented would be misleading. They are the explicit production extensions I would require for the supported payment rails.

Proposed path	Real-world case	Mandated behavior
CANCELLED	Merchant void or a valid authorization-reversal message after an order is abandoned	Authenticate and match the original reference; append cancellation; release only the remaining reservation once. Never create a principal refund for an unposted hold.
EXPIRED	Merchant never presents a capture before the applicable authorization deadline	Store a policy-derived deadline. A repeatable expiry job appends expiry and releases the remaining reservation. No universal expiry duration is assumed.
RELEASED	Operations confirms a duplicate, erroneous or orphaned hold	Require evidence and independent approval; record operator, reason and reference; release only the confirmed reservation. Do not alter an already settled posting.
SUPERSEDED	A merchant replaces an estimate with a revised authorization	Link old and new identities; test the resulting combined exposure and switch reservations atomically. Reject an unfunded increase without dropping the original protection.

Transitions that are easy to misclassify. Initial decline is a terminal decision on a request, not termination of an active hold. Release of the unused amount during final capture is part of settlement. A refund or chargeback after settlement is a new financial event, not reactivation of the old authorization. Fraud flags, a legal freeze or an account-closure request must not silently release a valid reservation into spendable funds: block or route the account until each obligation is resolved under the relevant policy.

Race and retry contract. Serialize hold changes per account and compare the authorization's expected version/state so settlement and cancellation/expiry cannot both release it. A duplicate network message should return the original outcome; a reused key with a different payload should raise a conflict. Today, duplicates prevent a second posting but return a new rejection instead of the original response. Retain terminal history with its external references.

Late presentment after expiry must enter an explicit exception flow governed by the payment rail's obligations. Do not resurrect the hold or assume that rejecting an unmatched capture removes the bank's external settlement liability. Supporting partial or multiple captures also needs a remaining-reservation model and final-capture indicator; the current single-final-settlement model does not provide them.

4. What was cut, why, and the risk deferred

These are the implemented core's material simplifications and the production obligations they defer. The in-memory/no-UI boundary was required by the exercise.

Simplification and reason	Production risk / next required decision
Memory-only, synchronous single writer; no service or UI	Restart loses state; concurrent writers can overspend. Require durable atomic commits, tested recovery and account-level serialization before service exposure.
Customer balances without double-entry counteraccounts; opening balances are configuration	Cannot prove balanced journals or reconcile cash, settlement, fee income and interest expense. Add a chart of accounts and balanced posting groups.
Fixed integer-day window; finalization cannot reopen	No time zones, cutoffs, holidays, value dates outside the window or closed-period adjustments. Introduce a controlled calendar and correction workflow.
Global scans, retained full history and full-report copies	Latency and resident memory grow with traffic; rejected attempts and installment fan-out also grow state. Add indexes, quotas, streaming and durable archival.
Two currencies, fixed pricing, no FX; BHD overdrafts fail closed	Valid debits may be refused; treating AED fees as BHD would misprice them. Require currency-specific approved product policies and versioned rates.
Principal-only, same-value-date, one-time reversal; fees retained	Cannot perform all bank-error redress, settlement corrections, partial refunds or chargebacks. Add reason-coded, linked adjustments and downstream reconciliation.
One final capture; no expiry, cancellation, release or replacement	Reservations can persist indefinitely and real network messages cannot be represented. Implement the lifecycle and race contract on page 3.
IDs rejected on any reuse; no payload-bound replay response	Network retries get ambiguous responses; identities disappear at restart. Store request identity, payload digest and original outcome durably.
Typed callers and limited runtime validation; no access controls	Malformed fields can throw; large inputs can exhaust resources. Add runtime schemas, input limits, caller authorization and audit identities.
Frozen objects instead of a durable audit/security boundary	A compromised process can forge history. Add access trails, protected storage, encryption, tamper detection and independent recovery/audit controls.
Final restated simple-interest projection and deterministic residual allocation	Not a general product-accrual engine; compounding, tiering, tax and customer presentation remain undefined. Approve conventions and preserve calculation/policy versions.
Rejections and reports only; no external settlement, redress, monitoring or reporting integrations	Local success cannot prove cash movement, customer notification or compliant returns. Reconcile external acknowledgements and operate exception queues with named owners.

Decision boundary. Preserve exact money, immutable sources, explicit temporal views and reproducible projections as product policy and operational controls are added. An append-only array does not supply those production controls by itself.